Natural Language as a Tactical Control Interface for Real-Time Strategy Games: An Engine-FSM Minimal MCP Approach Evaluated on OpenRA

JiZiYi
Universiti Teknologi Malaysia (UTM)
jiziyi@graduate.utm.my

Abstract

Real-time strategy (RTS) games coordinate dozens of mobile units across simultaneous fronts. Even when the player's tactical decision is clear, executing it through selection boxes, control groups, and queued orders is mechanically costly. We ask whether plain natural language can serve as the tactical control interface: can a player say "send the tanks down the middle and have the APCs flank from both sides" and have 30 mixed units execute the maneuver, without naming actor IDs, computing coordinates, or selecting from a 50-tool API menu? We present `openra_mcp`, a co-pilot for OpenRA — a production RTS engine — that exposes only two engine-side execution primitives (`Assault`, `Protection`) over the Model Context Protocol (MCP). The player issues free-form natural-language commands; an LLM translates them into structured squad calls; the engine runs per-tick movement, pathfinding, and combat. Higher-level tactics — pincer, multi-squad split, feint-and-raid, route constraints, time-sequenced attacks — are composed by the LLM on top of the two primitives rather than encoded as specialized engine FSMs. We evaluate the interface on three axes: (i) a ten-task natural-language capability suite (T1–T10) covering referent resolution, kind/state splitting, mid-flight re-commanding, partial cancel, conditional trigger, route constraint, formation, time-sequencing, and failure recovery, all passing; (ii) a live LLM session of eight free-form commands executed end-to-end; (iii) repeated DeepSeek-V4-Pro trials on three tactical scenarios (N=5 per scenario) showing low and stable LLM turn / token cost per command. We additionally report a cross-paradigm cost reference against OpenRA-RL, a per-unit atomic baseline, to characterise the architectural cost of atomic action APIs. Recent MCP-based UAV work shows that natural language to MCP-mediated multi-entity execution is emerging in robotics; this paper therefore claims the game/RTS branch of that design space rather than MCP swarm control in general. We position `openra_mcp` as the first engine-FSM-minimal MCP co-pilot for a production RTS — distinct from per-unit atomic-API designs used by both autonomous-player systems and an earlier open-source co-pilot effort (HOPPINZQ, 2025) — distinguish it from autonomous LLM-as-player systems, robotics swarm-control systems, and broad-surface co-pilots in turn-based or simulation games, and outline limitations (no formal user study) and a future-work path toward small local low-latency models distilled from LLM demonstrations.

1. Introduction

The mechanical cost of issuing a tactical decision in a real-time strategy (RTS) game often dwarfs the cost of making it. A player may decide in one second to split a force, route the armour through the middle, send the APCs around the flanks, delay the raiders by five seconds, and cancel only the third group; expressing that decision through selection boxes, control groups, queued orders, and rapid camera movement takes far longer and is error-prone, especially in large engagements with dozens of mixed units. This paper asks whether plain natural language — the same words the player would use to describe the plan to a friend — can serve as the tactical control interface for a production RTS, and what kind of engine-side abstraction makes that practical.

Most prior LLM-and-RTS research takes the **LLM-as-player** path: substituting the human with an agent and evaluating on game outcome. AlphaStar reached grandmaster StarCraft II play with reinforcement learning (Vinyals et al., 2019); recent LLM-based agents extend this line into language-driven strategic planning (Ma et al., 2024; Shao et al., 2024; Wang et al., 2023). Even systems that accept a natural-language prompt from a present human — most prominently HIVE (Anne et al., 2025) — typically run plan-once: the human issues a single instruction, the LLM emits a static plan, and the agents execute it autonomously until the engagement ends.

We pursue a different question — the **LLM-as-co-pilot** path — and a more specific design target: a *plain-natural-language* tactical interface for a real-time game with dozens of units, in which the player stays in the loop and the LLM is restricted to translation. The player never names actor IDs, computes coordinates, or selects from a fifty-tool menu; they speak the way they would describe the maneuver to a teammate. This direction has matured in code editing (GitHub Copilot, 2021; Cursor, 2023; Claude Code) and in flight simulators (Microsoft Flight Simulator Copilot, 2024), and exists in a pre-LLM form in games — Tom Clancy's EndWar (Ubisoft, 2008) demonstrated voice-commanded RTS with rule-based speech recognition more than fifteen years ago. At the same time, adjacent robotics work has begun to connect LLMs, MCP gateways, and drone systems, including UAV swarm mission execution (Ramos-Silva & Burke, 2026; Iannoli et al., 2026). We therefore do not claim to introduce MCP-mediated multi-entity control in general. The gap we target is narrower: to our knowledge, plain-language mixed-initiative co-piloting has not been instantiated for a production RTS engine with a real AI opponent using a modern LLM, and existing MCP-based game co-pilots target either turn-based games (STS2MCP for Slay the Spire 2; Gennadiyev, 2025) or simulation/sandbox games (oni_mcp for Oxygen Not Included; LightJUNction, 2026) — not real-time strategy.

We present `openra_mcp`, a prototype mixed-initiative co-pilot for OpenRA, an open-source engine that recreates and modernizes classic RTS games such as Red Alert and Command & Conquer (OpenRA Project, n.d.). The player issues natural-language commands such as "send the tanks down the middle and have the APCs flank from both sides" or "send the front group first, then launch the raiders five seconds later." The LLM maps the command to structured Model Context Protocol (MCP) tool calls. MCP provides a standard way to connect AI assistants to external tools and data systems (Anthropic, 2024); in this prototype, it is used as the bridge between the LLM client and an OpenRA unit control interface. The OpenRA engine then handles pathfinding, collision, attack-move behavior, and unit autonomy.

The current system is deliberately small on the engine side. After ablation, the LLM-facing game interface exposes two squad execution primitives: `Assault`, which pushes a unit set toward a target cell or actor, and `Protection`, which holds or defends a position. Higher-level tactics are not encoded as many specialized engine-side finite-state machines. They are composed above the engine boundary through Python/LLM logic and issued through MCP calls only when the tactical state changes.

This paper makes the following contributions:

1. **Plain natural language as the tactical control interface for a production RTS.** We demonstrate that a player can drive 30 mixed units through free-form natural-language commands — without naming actor IDs, computing coordinates, or selecting from a large tool menu — and have the engine execute the intended maneuver. To our knowledge this is the first such game/RTS co-pilot demonstration with an engine-FSM-minimal MCP surface on a real RTS engine with a real AI opponent — distinct from per-unit atomic-API designs in autonomous-player systems (OpenRA-RL Contributors, 2026) and an earlier open-source co-pilot effort (HOPPINZQ, 2025).
2. **Two-primitive engine-FSM design.** An ablation-justified game-side interface — `Assault` and `Protection` — sufficient to compose ten distinct natural-language tactical capabilities (referent resolution, kind/state splitting, mid-flight re-commanding, partial cancel, conditional trigger, route constraint, formation, time-sequencing, failure recovery), with the higher-level tactics composed on the LLM side rather than encoded as specialized engine FSMs.
3. **Paradigm distinction within the LLM-game and MCP-control space.** We separate *LLM-as-player* (autonomous game outcome) from *LLM-as-co-pilot* (human-in-loop intent translation) and further distinguish two co-pilot sub-philosophies — broad-surface (e.g. oni_mcp, ~320 tools for Oxygen Not Included) versus engine-FSM-minimal (this work, 17 tools plus 2 FSM primitives) — while explicitly separating game/RTS co-piloting from adjacent MCP-mediated robotics swarm control.
4. **Multi-axis evaluation on the same LLM.** A ten-task NL capability suite passing 10/10, a live LLM session of eight free-form commands, and repeated DeepSeek-V4-Pro trials on three tactical scenarios (N=5 per scenario) with per-run logs.
5. **Cross-paradigm cost reference.** Under the same LLM, the per-unit atomic baseline (OpenRA-RL) requires 40 ± 7 LLM responses and 755 ± 218k tokens (N=3 full games) to reach a terminal state; this is provided as an architectural cost reference, not a head-to-head gameplay benchmark.
6. **Open and reproducible artifacts.** Engine fork, MCP server, capability suite, runners, and raw per-turn logs are released. To our knowledge this is the first open-source RTS co-pilot accompanied by a paper, ablation, and multi-axis repeated evaluation; an earlier open-source effort (HOPPINZQ, 2025) adopts a per-unit atomic API without a paper or controlled evaluation.
7. **A future-work path** toward small local models distilled from high-quality LLM demonstrations, emitting low-latency MCP command JSON to reduce per-command LLM latency for real-time play.

2. Related Work

Prior research on language-driven RTS systems falls into two distinct paradigms that share surface mechanics but optimize for different objectives:

- **Paradigm A — LLM-as-player.** A language model (alone or hybridized with RL) replaces the human player and is evaluated on game-outcome metrics (win rate, score, reward). AlphaStar, TextStarCraft II, SwarmBrain, HIMA, OpenRA-RL, and HIVE all sit in this paradigm.
- **Paradigm B — LLM-as-co-pilot.** A language model translates a present human player's natural-language intent into engine-level actions; the human retains strategy, economy, and battlefield interpretation. Evaluation targets the translation cost (turns, tokens, latency) and intent fidelity, not game outcome.

`openra_mcp` is positioned in Paradigm B. Conflating the two paradigms invites mismatched evaluation criteria (e.g. asking a co-pilot for a win rate, or asking an autonomous agent for prompt-fidelity); we treat all Paradigm A systems as comparison points on the *cost* axis only, not as precedents.

2.0 Game + MCP landscape (May 2026): We surveyed open-source MCP servers connected to interactive games via GitHub (keyword `mcp game`, sorted by stars, ≥40 stars retained, 25 repositories examined). The results cluster into four groups:

Group	Representative repositories	Stars (≥40 each)	Position vs `openra_mcp`
(i) Game-development tools for designers	`Coding-Solo/godot-mcp` (3.8k), `ee0pdt/Godot-MCP` (570), `AnkleBreaker-Studio/unity-mcp-server` (199, 268 tools), `tugcantopaloglu/godot-mcp` (219, 149 tools), `MubarakHAlketbi/game-asset-mcp` (137), `youichi-uda/unity-mcp-pro-plugin` (57, 147 tools), `HurtzDonutStudios/ai-forge-mcp` (61, 565 tools)	—	Different user (developer, not player); orthogonal
(ii) Autonomous in-game agents (Paradigm A)	`Gennadiyev/STS2MCP` (363, Slay the Spire 2), `CharTyr/STS2-Agent` (243), `yuniko-software/minecraft-mcp-server` (586), `notpoiu/roblox-executor-mcp` (60), `Whale-io/lets-play-a-game` (132)	—	Same MCP infrastructure, different paradigm
(iii) Emulator / debug / reverse engineering	`drhelius/Gearboy` (1.1k, Game Boy emulator + MCP debug), `drhelius/Gearsystem` (369), `0xhackerfren/frida-game-hacking-mcp` (65), `bethington/cheat-engine-server-python` (49)	—	Non-gameplay infra
(iv) Co-pilot for human players (Paradigm B)	`LightJUNction/OniMods/oni_mcp` (Oxygen Not Included), `openra_mcp` (this work)	2 repositories total	Direct paradigm peers

Group (iv) — the design space this paper targets — is sparse. The two repositories represent two distinguishable sub-philosophies within Paradigm B:

- **Broad-surface co-pilot.** `oni_mcp` exposes approximately 320 tools spanning colony state, building configuration, scheduling, automation, research, and sandbox controls, with a small DSL-style batch wrapper (`agent_program_execute`). The LLM is expected to navigate a wide tool catalogue and decompose tactics itself. This fits Oxygen Not Included's low time pressure (a base-building simulation), where LLM latency does not block gameplay.
- **Engine-FSM-minimal co-pilot.** `openra_mcp` exposes 17 tools, of which two (`spawn_squad`, `spawn_squad_batch`) trigger engine-side FSMs that execute per-tick movement, autotargeting, and cohesion internally. The LLM issues a task-level command, then the engine runs the maneuver without further LLM calls until the player or an event changes the intent. This fits real-time strategy, where per-tick LLM control would exceed the game's clock rate.

These philosophies are complementary rather than competing — they suit different game-time-pressure regimes. We adopt the engine-FSM-minimal approach because the target game is real-time and the per-tick budget for LLM round-trips is effectively zero.

2.1 Cross-domain MCP and swarm-control precedents: Recent robotics work provides an important boundary condition for this paper: natural language to MCP-mediated actuation is no longer unique to games. Ramos-Silva and Burke (2026) propose

a universal LLM drone command-and-control interface using MCP over MAVLink/MAVSDK for real and simulated drones. Even closer, Iannoli et al. (2026) present a Web-of-Drones framework in which a user states a UAV swarm mission in natural language and an LLM Agent Core interacts through an MCP gateway and Web-of-Things abstractions to execute area coverage, formation, and smart-irrigation missions in ArduPilot-based simulation.

These systems are strong adjacent precedents, and they rule out a broad claim that `openra_mcp` is the first LLM/MCP system for controlling many entities. Our claim is narrower and game-specific. `openra_mcp` targets a production RTS rather than a robotics swarm simulator; the controlled entities are heterogeneous combat units rather than UAV platforms; the user is an active player who retains strategy, economy, and battlefield judgment rather than a mission designer who leaves the system to complete the task; and the control layer emits squad-level game primitives rather than physical-device actions. The relationship is therefore complementary: robotics work establishes a cross-domain MCP multi-entity precedent, while this paper studies the RTS game-control branch of the same larger design space.

2.2 Cross-domain co-pilot precedents (Paradigm B lineage): The mature reference point for LLM co-pilots is not games at all but software engineering. GitHub Copilot, Cursor, and Claude Code translate developer natural-language intent into code edits and tool calls while the human retains goal selection, design judgment, and final acceptance authority. The same pattern appears in Microsoft Flight Simulator's voice copilot demonstration (2024) for cockpit operations and in OpenAI's ChatGPT Code Interpreter for data-analysis workflows. These systems share three properties with `openra_mcp`: (i) the human stays in the loop and owns the goal, (ii) the LLM emits structured tool calls auditable by the human, and (iii) the underlying engine — compiler, OS, simulator — owns the execution loop. To our knowledge, this paradigm has not previously been instantiated against a production RTS engine with a real opponent AI.

2.3 NL game control before LLMs: Pre-LLM RTS work demonstrated that natural-language unit control is desirable but constrained by available NL technology. Tom Clancy's EndWar (Ubisoft, 2008) used speech recognition with a fixed grammar to issue squad-level commands ("Bravo, attack hostile two") and remains the most prominent commercial example of voice-controlled RTS. Black & White (Lionhead, 2001) used gesture and voice as primary input for a god game. These systems established the design intent — humans command, the engine executes — but were limited to rule-based parsers. `openra_mcp` continues this lineage with modern LLM-based natural-language understanding.

2.4 Paradigm A — LLM-as-player in RTS (compared, not precedent): AlphaStar reached elite human performance in StarCraft II via reinforcement learning, substituting the human player with an agent (Vinyals et al., 2019). TextStarCraft II converts StarCraft II into a text-based benchmark for evaluating LLM strategic decision-making and introduces a chain-of-summarization approach for real-time strategic settings (Ma et al., 2024). SwarmBrain uses an LLM-powered high-level module with a lower-level reflex layer for StarCraft II, separating strategic reasoning from faster tactical reactions (Shao et al., 2024). HIMA extends this line through a hierarchical imitation multi-agent framework for StarCraft II, with specialized imitation agents orchestrated by a strategic planner (Ahn et al., 2025). OpenRA-RL frames Red Alert as a platform for scripted bots, reinforcement learning, and LLM players via a 48-tool atomic action API (OpenRA-RL Contributors, 2026); we use it as our direct cost-axis baseline in §6. An earlier open-source RTS effort, `hoppinai-mcp-red95` (HOPPINZQ, 2025), adopts the same per-unit atomic API style but in Paradigm B (co-pilot) mode — illustrating that the atomic-vs-task-level architectural choice is orthogonal to the autonomy axis.

HIVE deserves separate treatment because it is frequently miscategorized as a co-pilot system. HIVE enables a single human to coordinate swarms of up to 2,000 agents through natural-language dialogue with an LLM, generating behavior-tree-like plans for a custom multi-agent benchmark (Anne et al., 2025). The architecture, however, is plan-once: the LLM is invoked once per human command and emits a static plan executed for the remainder of the engagement. The agents are JAX-vectorized abstract particles rather than RTS units with pathfinding, weapon systems, or a learning AI opponent; the enemy side is hardcoded. The code for the LLM-to-DSL planner and the benchmark engine is not publicly released at the time of writing. HIVE is therefore better characterized as an *adjacent* Paradigm A demonstration that uses NL prompts as a configuration interface to a planner, rather than as a closed-loop co-pilot for a production RTS. `openra_mcp` differs along all of these axes: production engine, closed-loop event-driven LLM invocation, real AI opponent, open-source implementation.

2.5 Engine-level autonomy abstractions: Behavior trees and hierarchical task networks are the canonical engine-level abstractions for decomposing high-level commands into per-tick behavior (Champandard, 2007). HIVE's five behavior trees follow this lineage; our two-primitive squad FSM (Assault + Protection) is a deliberately minimal specialization of the same idea, validated empirically by ablation showing that ten natural-language tactical capabilities (T1-T10) compose from these two primitives alone (§6.4). Real-time human-AI coordination systems such as HLA address LLM latency by separating slow language reasoning, faster macro-action generation, and reactive execution (Liu et al., 2023). Our architecture aligns with this principle: the LLM should not sit in a high-frequency loop; it should translate intent and compose task-level actions while lower layers handle per-tick execution.

2.6 Mixed-initiative HCI and tool-use frameworks: The mixed-initiative framing comes from earlier HCI work arguing that useful interfaces require careful sharing of initiative between humans and agents rather than simple automation (Horvitz, 1999). More recent guidelines for human-AI interaction emphasize making clear what the system can do, how uncertainty and failure are handled, and when the user remains in control (Amershi et al., 2019). `openra_mcp` applies these ideas to RTS control by

assigning strategic judgment, economy, and battlefield interpretation to the player while assigning tactical translation to the LLM. Embodied LLM agents such as Voyager demonstrate the power of tool-using language models in game-like environments (Wang et al., 2023). The Model Context Protocol (Anthropic, 2024) provides the tool-call infrastructure used by `openra_mcp` to expose the squad primitives to the LLM. The resulting research question is therefore not "Can an LLM play Red Alert?" but "Can natural language become a reliable tactical control layer for a human player of a real-time strategy game?"

2.7 Position summary:

Line of work	Paradigm	Environment	Human role	LLM control layer	Status as precedent for `openra_mcp`
AlphaStar (Vinyals et al., 2019)	A	StarCraft II	Spectator/opponent	Learned full-game RL policy	Comparison only — different paradigm
TextStarCraft II / HIMA (Ma 2024; Ahn 2025)	A	Textual or structured SC2	Outside the loop	Strategic LLM planning	Comparison only — different paradigm
SwarmBrain (Shao et al., 2024)	A	StarCraft II	Outside the loop	LLM strategy + reflex layer	Comparison only — different paradigm
OpenRA-RL (Contributors, 2026)	A	OpenRA, atomic 48-tool API	Outside the loop	Per-unit atomic actions	Direct *cost-axis* baseline in §6, not paradigm precedent
HIVE (Anne et al., 2025)	A (adjacent)	JAX swarm sim, plan-once	Single NL prompt, then absent	LLM emits static DSL plan	Closest surface, but plan-once + toy engine + closed source — not a Paradigm B precedent
Universal LLM Drone C2 (Ramos-Silva & Burke, 2026)	Adjacent robotics	Real/simulated drones	Operator gives drone command	MCP over MAVLink/MAVSDK	MCP physical-control precedent, not game/RTS
Web-of-Drones swarm (Iannoli et al., 2026)	Adjacent robotics	ArduPilot UAV swarm simulation	Gives mission objective, then absent during execution	LLM Agent Core + MCP gateway + WoT	Closest non-game MCP multi-entity analogue; not RTS/mixed-initiative game control
Copilot / Cursor / Claude Code	B (other domain)	Code editor	Owns design, accepts edits	NL-to-edit + tool calls	Cross-domain precedent
EndWar (Ubisoft, 2008)	B (pre-LLM)	Commercial RTS	Owns strategy, voice commander	Rule-based ASR + grammar	Lineage precedent
Hoppinai (HOPPINZQ, 2025)	B	OpenRA	Owns strategy, issues per-unit commands	Per-unit atomic API (move/attack by actor_id)	Earlier open-source RTS co-pilot effort; atomic-API design choice (cf. OpenRA-RL); no paper or controlled evaluation

Line of work	Paradigm	Environment	Human role	LLM control layer	Status as precedent for `openra_mcp`
`openra_mcp` (this work)	**B**	**OpenRA, production RTS**	**Owns strategy/economy/judgment**	**Two-primitive squad FSM + LLM composition**	**First engine-FSM-minimal RTS co-pilot with paper, ablation, and multi-axis evaluation**

3. System Overview

The system has four layers:

```
Human player intent
        ->
LLM tactical translator
        ->
MCP tool calls
        ->
OpenRA squad execution primitives
        ->
Engine unit autonomy
```

The human layer supplies strategic and tactical intent in natural language. The LLM layer translates that intent into structured tool calls. The MCP layer provides an auditable interface between the LLM client and the game-side server. The OpenRA layer executes squad-level primitives and delegates local movement and combat details to the engine.

The design follows an explicit authority split:

- The player owns strategic judgment.
- The player owns economy and production decisions.
- The player owns battlefield interpretation.
- The LLM owns tactical translation and composition.
- The engine owns pathfinding, collision, and local unit behavior.

This authority split is central to the paper. The goal is not to make an AI that plays the game for the user. The goal is to make an AI co-pilot that turns the user's declared tactical intent into complex executable control.

The architecture also creates an audit trail. A natural-language command is not converted into invisible mouse input. It becomes a structured MCP tool call, such as `spawn_squad` or `spawn_squad_batch`, with explicit unit identifiers, target positions, and squad types. This matters for research reproducibility: when a command succeeds or fails, the intermediate representation can be inspected, compared across runs, or used later as a supervised training target for smaller models.

3.1 Example Translation: The following simplified example illustrates the intended interface. A player utterance is first interpreted at the tactical level and then emitted as a structured MCP call:

```
{
  "player_command": "Tanks through the middle, APCs flank from both sides",
  "mcp_tool": "spawn_squad_batch",
  "arguments": {
    "squads": [
      {
        "squad_type": "Assault",
        "unit_ids": [101, 102, 103],
        "target_pos": { "x": 60, "y": 50 }
      },
      {
        "squad_type": "Assault",
        "unit_ids": [201, 202, 203],
```

```
        "target_pos": { "x": 60, "y": 40 }
      },
      {
        "squad_type": "Assault",
        "unit_ids": [301, 302, 303],
        "target_pos": { "x": 60, "y": 60 }
      }
    ]
  }
}
```

The exact unit identifiers and target cells come from the current game state and the composition layer. OpenRA then executes the squad orders through its own pathfinding, collision, and combat systems. The LLM does not issue per-tick movement commands.

4. Two Execution Primitives

Early versions of the project explored a larger daemon and DSL surface. After the v2 tactical capability suite, the implementation was reduced. The current LLM-facing design relies on two squad execution primitives.

`Assault` pushes a set of units toward a target cell or actor. It is the core primitive for attacks, movement, pincer arms, route-constrained pushes, and timed raids. `Protection` holds a position and defends a target cell. These primitives are intentionally execution-level, not strategic. They do not decide what the player should build, whether a fight is favorable, or which strategic objective matters.

Higher-level tactics are composed outside the engine primitive layer. For example, pincer movement is represented as two Assault squads approaching from different sides. A time-sequenced attack is represented as one squad launched first and a second squad launched after a delay. A feint-and-raid plan is represented as separate squad calls with different targets and timing.

This design keeps the game-side interface small and makes the LLM's output auditable: each player command can be inspected as a sequence of MCP calls over a small vocabulary.

5. Tactical Composition From Natural Language

The prototype demonstrates several forms of tactical composition:

- referential control: "the left group" versus "the right group";
- kind-based splitting: tanks through the middle, APCs on the flanks;
- state-based splitting: damaged units return, healthy units continue;
- mid-flight recommanding: stop one ongoing movement and replace it;
- partial cancellation: recall one subgroup while others continue;
- conditional behavior: retreat if a condition is met;
- route constraints: avoid the center and move through the side;
- formation-like behavior: tanks ahead, APCs behind;
- time sequencing: launch the main force first, raiders later;
- failure recovery: re-plan when a group is stuck.

These are not merely textual suggestions. The LLM produces tool calls that the OpenRA bridge executes in the game engine.

This section is also the bridge to the local-model future work. The target output is not an unstructured essay and not a hidden policy. It is a compact JSON-like action specification over a small tool vocabulary. That makes the problem suitable for supervised fine-tuning: large models can generate high-quality demonstrations, while smaller local models can be trained to imitate the mapping from a compact game-state summary plus player instruction to MCP calls.

6. Evaluation

The current evidence base contains four parts: a natural-language tactical capability suite, a live LLM demonstration, earlier two-primitive tactical demonstrations, and an ablation showing that unused daemon/DSL machinery could be removed without losing the demonstrated tactical path.

6.1 Natural-Language Tactical Capability Suite: The v2 capability suite tests ten tactical capabilities that are difficult to express as a single conventional RTS command. Each scenario begins with an English or Chinese natural-language instruction and evaluates whether the system selects the correct units, decomposes the task, and satisfies the tactical intent in OpenRA. Table 1 reports the final merged result across the base post-ablation run and targeted retry files. This is intentionally reported as a merged final result: the raw `v2_post_ablation.csv` file alone contains threshold/scenario failures for T3, T5, T7, and T8, and the corrected outcomes come from `v2_retry_4fails.csv` and `v2_t7_retry2.csv`.

ID	Capability	Natural-language command	Units	Subtasks	Final result	Latency
T1	Referential resolution	“Let the left group flank; the right group stays still”	80	1	Pass	348 ms
T2	Kind-based split	“Tanks through the middle, APCs flank from both sides”	100	3	Pass	356 ms
T3	State-based split	“Damaged units return; the rest keep pushing”	64	2	Pass	360 ms
T4	Mid-flight re-command	“Everyone push, stop, then switch to two-side pincer”	100	3	Pass	8,736 ms
T5	Partial cancel	“The third group returns; the others continue”	64	6	Pass	3,636 ms
T6	Conditional retreat	“If enemy main force appears, retreat to the bridge; otherwise continue”	75	2	Pass	1,712 ms
T7	Route constraint	“Group A goes directly; group B goes far right first”	64	2	Pass	25,175 ms
T8	Formation-like movement	“Tanks in front, APCs behind; do not clump”	64	2	Pass	310 ms
T9	Time sequencing	“Main force first; raiders attack from the side five seconds later”	75	2	Pass	5,362 ms

ID	Capability	Natural-language command	Units	Subtasks	Final result	Latency
T10	Failure recovery	“If a group gets stuck, re-plan the route”	75	1	Pass	20,657 ms

The final merged suite reaches 10/10 pass. The longer latencies in T4, T7, and T10 reflect multi-stage or waiting behavior rather than only the initial tool call time.

6.2 Live LLM Demonstration: The live demonstration records Claude receiving player natural language, reading game state, selecting units, and emitting `spawn_squad` or `spawn_squad_batch` calls. The demonstration contains eight consecutive commands with no failed command in the recorded sequence.

Step	Player intent	Capability shown
1	Move all units to the lower-right area	100-unit single-squad movement
2	Send APCs to upper-left while one tank stays still	Unit-kind split and local exception
3	Send the left half downward and the right half upward	Spatial reference resolution
4	Regroup at center, then split into four teams to four corners	Multi-stage regroup and four-way split
5	Cycle four teams around the map for 180 seconds	Long-running composed patrol
6	Regroup all units at lower-left	Global recall/regroup
7	Send a 20-unit team first, then 80 units after eight seconds	Time-sequenced small/large force coordination
8	Execute a pincer around the central building	50/50 split and converging attack arms

This sequence is important because it uses a live LLM rather than a fixed script: the model reads `get_state`, selects unit IDs, computes or chooses targets, and emits batch JSON.

6.3 Two-Primitive Demonstrations: Earlier E7 demonstrations validate the engine-side primitive path. In those runs, `spawn_squad_batch` dispatched four squads in 31-34 ms, an eight-squad batch in about 10 ms, and a single 60-unit squad in about 18 ms. Demonstrated tactics include single-squad assault, four-corner batch movement, composed patrol, pincer movement, feint plus raid, mixed APC/tank squads, combined-arms offset movement, large-unit movement, and eight-direction batch movement.

These demonstrations support the architectural claim that complex tactics do not require a large set of engine-side tactical FSMs. A small execution layer can support richer behavior when the composition logic lives above it.

6.4 Ablation and Telemetry: The phase ablation removed the unused daemon and DSL path while retaining the `spawn_squad` / `spawn_squad_batch` execution route. The active MCP surface was reduced from 31 tools to 17 tools, and the Python implementation shrank by approximately 6,700 lines. The post-ablation capability path remained viable, which supports the “two engine primitives plus LLM-side composition” design.

Development-session telemetry provides a rough observability check rather than a controlled user-study result. Across existing `decisions.jsonl` logs, `paper_metrics.py` found 123 decision records, 117 successful decisions, 6 errors, 98 commands with recorded natural-language input, and 92 unique natural-language inputs. Across Claude transcript logs for this project, it found 781 OpenRA MCP tool events. The median game-side decision latency was 140 ms, the median logged LLM latency field was 600 ms, and the median time from a user text turn to the first OpenRA MCP tool event was about 7.19 seconds. Because these logs include development and debugging sessions, these numbers should be read as prototype telemetry, not as final benchmark results.

6.5 Repeated DeepSeek-V4-Pro tactical trials (N=5 per scenario): To probe how stable the LLM translation cost is across repetitions and how it scales with roster size, we re-ran three tactical scenarios five times each at two roster sizes — 30 units (12 e1 + 8 3tnk + 6 v2rl + 4 apc) and 100 units (40 e1 + 30 3tnk + 20 v2rl + 10 apc) — through a same-game OpenRA

session driven by DeepSeek-V4-Pro: *scen1* — push all mobile units to map bottom-right (78, 85); *scen2* — split into four squads, push to four corners; *scen3* — two-phase 50/50 pincer onto map centre. Between runs, surviving units were recalled to a rally point near the player base via a one-shot `spawn_squad`. Each scenario carries its own verification window before unit positions are checked; the verify radius and per-corner unit threshold scale with √(roster / 30) to account for on-arrival congestion at larger roster sizes.

30-unit trials (N=5 per scenario):

Scenario	Success	LLM turns mean ± std (min, max)	Total tokens mean ± std (min, max)	Wallclock mean ± std (s)
scen1 full push BR	4/5	3.0 ± 0.0 (3, 3)	7,136 ± 106 (6,999, 7,296)	77.9 ± 4.8
scen2 4-corner split	4/5	2.6 ± 0.89 (1, 3)	7,009 ± 3,360 (1,028, 8,958)	108.4 ± 54.9
scen3 50/50 pincer	2/5	5.6 ± 2.51 (2, 8)	20,502 ± 11,535 (5,182, 35,038)	206.5 ± 102.7

100-unit trials (N=5 per scenario, 3.3× larger roster):

Scenario	Success	LLM turns mean ± std (min, max)	Total tokens mean ± std (min, max)	Wallclock mean ± std (s)
scen1 full push BR	0/5	3.8 ± 0.84 (3, 5)	19,752 ± 10,195 (10,898, 35,881)	154.4 ± 8.7
scen2 4-corner split	0/5	3.6 ± 0.55 (3, 4)	18,407 ± 5,846 (11,776, 23,376)	186.9 ± 3.4
scen3 50/50 pincer	0/5	5.8 ± 0.84 (5, 7)	33,534 ± 11,836 (22,264, 50,086)	305.0 ± 9.1

LLM-side cost scales sub-linearly with roster. Roster grew 3.3×; LLM turns grew only 1.04–1.38× across the three scenarios. Token usage grew 1.6–2.8×, the increase driven almost entirely by larger observation payloads (longer unit lists in `get_state`) rather than by additional LLM round-trips.

100-unit verification did not pass the success criterion, but NL-to-action translation remained correct. In every 100-unit trial, the LLM produced syntactically valid tool calls matching the requested tactic (e.g. scen2 r01–r05 all emitted a four-squad batch with balanced allocations of approximately 25 units each, and the engine reported post-execution distributions of [14, 14, 26, 14] units per corner — 80 out of 100 units traversed correctly, but the per-corner ≥15-unit threshold required all four corners filled). scen1 100-unit consistently delivered 56/100 units to within radius 18 of the target, falling 4 units short of the 60% threshold. scen3 100-unit suffered from engine-level congestion during the phase-1-to-phase-2 transition, delivering 0–28 units to the centre depending on traversal timing. These are engine-side execution outcomes, not translation failures, and we report them transparently rather than relaxing the criterion.

30-unit per-scenario failure modes (N=15 total): one DeepSeek transient connection drop (scen2 r03), one DeepSeek malformed JSON in a tool call (scen3 r01), one timing edge where the verify window fired before units reached the target (scen1 r02), and two squad-overlap interactions in scen3 (r02 / r03) where the LLM issued the phase-2 `spawn_squad_batch` without first calling `cancel_squad` on the phase-1 squad, leaving the phase-1 squad still holding the units. The squad-overlap interaction is a real system limitation documented in the project's design notes; in scen3 r04 / r05 the LLM issued explicit `cancel_squad` calls between phases and succeeded.

Per-call patterns are stable across both roster sizes. Each scenario typically consists of one `get_state` read, one `spawn_squad` (or `spawn_squad_batch`) dispatch, and an optional `wait` or `done`. scen3 adds one extra dispatch and one `wait` for the second phase. The LLM never fabricated a tool name, never produced a tool argument the engine rejected as malformed, and consistently produced balanced unit allocations across split-target requests.

Raw per-run data: `logs/rl_compare/our_deepseek_results_runs_n30.csv` and `logs/rl_compare/our_deepseek_results_runs`
Summaries: `our_deepseek_summary_n30.csv`, `our_deepseek_summary_n100.csv`.

6.6 Cross-paradigm cost reference: per-unit atomic baseline: To characterise the architectural cost of atomic per-unit MCP APIs — not to make a head-to-head gameplay claim — we ran DeepSeek-V4-Pro through the OpenRA-RL agent harness (Contributors, 2026), which exposes 48 atomic tools (`move_units`, `attack_target`, `build_unit`, `advance`, etc.) over a docker-hosted OpenRA fork. Three independent full games were run.

Run	LLM responses	Tool calls	Prompt tokens	Completion tokens	Total tokens	Wallclock (s)	Outcome
1	47	78	980,323	13,940	994,263	518.9	LOSE
2	33	55	550,221	9,028	559,249	440.8	LOSE
3	40	66	699,546	11,585	711,131	556.0	LOSE
mean ± std	**40.0 ± 7.0**	**66.3 ± 11.5**	**743,363 ± 218,540**	**11,518 ± 2,462**	**754,881 ± 217,924**	**505.2 ± 58.9**	**3/3 LOSE**

Average prompt size per response across runs is ~18–21k tokens, dominated by the accumulating conversation history and the 48-tool schema repeated on every call. Average latency per LLM call is 11.0–14.7 s. All three games reached LOSE without exhausting the 100-turn / 1800-second cap.

> **Caveat — this is a cost reference, not a benchmark.** The OpenRA-RL run plays a full game (economy + combat + outcome), while our `openra_mcp` scenarios in §6.5 are isolated tactical commands on a pre-trained roster. Player-owned economy is by design in `openra_mcp` (see §3), so the workloads are not apples-to-apples; we therefore do not claim a direct gameplay or efficiency win over OpenRA-RL. The numbers serve a different purpose: they illustrate the per-turn prompt-size and total- token implications of exposing per-unit atomic actions as the LLM's control surface, which is the architectural choice this paper argues against for real-time strategy.

For context, a same-engine scripted baseline (no LLM, the official `ScriptedBot` driving identical isolated tactical tasks) requires 3,369 ± 9 environment round-trips per scenario set across three runs. This is an action-granularity baseline that quantifies how much repeated control work an atomic engine API would push onto any controller — LLM or otherwise — that has not been promoted to a task-level abstraction.

Raw logs and summaries: `logs/rl_compare/`.

7. Discussion

The main lesson is that LLMs do not need to operate at the same level as traditional game-control agents. They are poorly matched to high-frequency per-tick control and fragile numerical battlefield prediction. They are better matched to translating natural language into structured, auditable, task-level commands.

The co-pilot framing also changes what the system should and should not do. A traditional autonomous agent may need to estimate whether an attack is likely to succeed. This system should not. The player sees the game, makes the strategic judgment, and decides what they want. The AI co-pilot should execute the declared tactical intent faithfully.

The project also suggests a plausible path toward local models. The output space is structured MCP JSON over a small tool vocabulary. This is a better distillation target than open-ended strategic reasoning. A large model can generate high-quality demonstrations, and a smaller model can later be trained to produce valid MCP calls under lower latency and cost.

8. Limitations

This is an early prototype and should be presented as such.

No formal user study. All evaluations in §6 are LLM-driven from scripted natural-language commands, not from human players unfamiliar with the system. We do not measure player learning curve, subjective ease-of-use, recovery from misinterpretation, or per-task wallclock versus manual mouse-and-keyboard play. Claims of "plain natural language" are supported by the breadth of capability tests (T1–T10, spanning referent resolution, kind/state splitting, mid-flight re-commanding, partial cancel, conditional trigger, route constraint, formation, time-sequencing, failure recovery) and by a live free-form LLM session, but a controlled human-subjects study remains future work (see §9).

Small sample sizes. The repeated DeepSeek trials are N=5 per scenario; the cross-paradigm cost reference is N=3 full games. These are sufficient to characterise central tendency and basic variance but not for strong statistical claims. We report mean ± standard deviation and the per-run table to allow readers to assess noise directly.

Workload asymmetry in the cost reference. The OpenRA-RL cost reference in §6.6 plays a complete game (economy + combat + outcome), whereas the openra_mcp scenarios in §6.5 execute isolated tactical commands on a pre-trained roster.

This is by design — player-owned economy is a paradigm constraint, not an evaluation oversight — but makes the two cost numbers not directly comparable as a win-rate or end-to-end-efficiency benchmark.

Scenario coverage. Tactical scenarios use prepared OpenRA setups rather than full competitive matches; sandbox conditions and `/instantbuild` cheats accelerate roster preparation. Evaluation has been conducted on the Soviet faction; Allied units (1tnk/2tnk/medi/mech) have not been systematically retested post-ablation.

Engine-side limitations. The squad-overlap interaction documented in §6.5 — when the LLM dispatches a new squad without first cancelling the holding squad — is a real system limitation, not a translation failure. Improving the engine to either reassign units on overlap or surface the conflict to the LLM is engineering work.

Information boundary. Player-owned-information and player-owned- economy boundaries are currently enforced by protocol and tool surface design, not by a complete formal security mechanism.

Local small model not yet trained. The future-work distillation path is unrealised in this paper.

Telemetry mixes sources. Development-session telemetry (§6.4) includes debugging and authoring sessions and should be read as prototype observability, not benchmark data.

9. Future Work

Future work should proceed in four directions. First, run repeated clean evaluations across maps, factions, and enemy conditions. Second, compare the co-pilot against manual control using click count, completion time, and error rate. Third, extract supervised training pairs from LLM demonstrations and fine-tune a small local model for low-latency MCP command generation. Fourth, strengthen the boundary between player-owned economic/information authority and AI-owned tactical translation.

10. Conclusion

`openra_mcp` demonstrates a practical early version of natural-language tactical control for RTS games. The system does not try to make the LLM a complete player. Instead, it keeps the human in charge of strategy, economy, and battlefield judgment, while the LLM translates tactical intent into auditable MCP calls over a small set of squad execution primitives. The current prototype evidence is preliminary but concrete: complex OpenRA maneuvers such as splitting, pincer movement, route constraints, time sequencing, and failure recovery can be expressed as player language and executed in the game.

The broader implication is that language models may be most useful in games when they are placed at the right level of abstraction. They need not drive every unit every tick. For RTS co-piloting, the more promising role is to turn human tactical intent into structured, executable control while the engine handles local behavior. This framing also creates a clear path toward small local models: the learning target is not full autonomous strategy, but valid low-latency MCP command generation from player intent and compact game state.

Data and Code Availability

The prototype code, experiment scripts, logs, and draft manuscript are maintained in the `openra_mcp` project directory. The evaluation evidence used in this preprint is drawn from:

- `mcp_server/experiments/scenarios_v2.py`
- `logs/v2_post_ablation.csv`
- `logs/v2_retry_4fails.csv`
- `logs/v2_t7_retry2.csv`
- `logs/baseline_pre_ablation.md`
- `logs/live_llm_demo/demo_01.mp4`
- `logs/v2_videos/*.mp4`
- `logs/paper_metrics.json`

The first public release should include the manuscript, the relevant CSV files, the code snapshot, and either the videos themselves or stable links to the video supplement.

Ethics and Boundary Statement

This project is a game-control prototype and does not involve human-subjects data collection in its current evaluation. The main ethical issue is not player privacy but control delegation. The system is therefore designed around an explicit boundary: the player owns strategy, economy, and battlefield judgment, while the LLM translates tactical intent into executable actions. The paper does not claim that the system should make strategic decisions for players or that it can predict whether an engagement is favorable.

AI Assistance Disclosure

This manuscript and its supporting metadata were drafted with AI assistance. The author provided the project, design intent, implementation evidence, and publication goal. AI assistance was used to organize the argument, draft prose, prepare tables from local logs, and assemble release metadata. The author is responsible for verifying the final claims, references, and release artifacts.

References

Ahn, D., Kim, S., & Choi, J. (2025). *Society of mind meets real-time strategy: A hierarchical multi-agent framework for strategic reasoning*. arXiv:2508.06042. https://arxiv.org/abs/2508.06042

Amershi, S., Weld, D., Vorvoreanu, M., Fourney, A., Nushi, B., Collisson, P., Suh, J., Iqbal, S., Bennett, P. N., Inkpen, K., Teevan, J., Kikin-Gil, R., & Horvitz, E. (2019). Guidelines for human-AI interaction. In *Proceedings of the 2019 CHI Conference on Human Factors in Computing Systems* (pp. 1-13). https://doi.org/10.1145/3290605.3300233

Anne, T., Syrkis, N., Elhosni, M., Turati, F., Legendre, F., Jaquier, A., & Risi, S. (2025). *Harnessing language for coordination: A framework and benchmark for LLM-driven multi-agent control*. arXiv:2412.11761. https://arxiv.org/abs/2412.11761

Anthropic. (2024). *Introducing the Model Context Protocol*. https://www.anthropic.com/research/model-context-protocol

Horvitz, E. (1999). Principles of mixed-initiative user interfaces. In *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems* (pp. 159-166). https://doi.org/10.1145/302979.303030

HOPPINZQ. (2025). *hoppinai-mcp-red95: Conversational Red Alert via MCP*. Open-source software, first commit 2025-10-29. https://github.com/HOPPINZQ/hoppinai-mcp-red95

Iannoli, A., Gigli, L., Sciullo, L., Trotta, A., & Di Felice, M. (2026). *Say the mission, execute the swarm: Agent-enhanced LLM reasoning in the Web-of-Drones*. arXiv:2605.03788. https://arxiv.org/abs/2605.03788

Liu, J., Yu, C., Gao, J., Xie, Y., Liao, Q., Wu, Y., & Wang, Y. (2023). *LLM-powered hierarchical language agent for real-time human-AI coordination*. arXiv:2312.15224. https://arxiv.org/abs/2312.15224

Ma, W., Mi, Q., Zeng, Y., Yan, X., Wu, Y., Lin, R., Zhang, H., & Wang, J. (2024). *Large language models play StarCraft II: Benchmarks and a chain of summarization approach*. arXiv:2312.11865. https://arxiv.org/abs/2312.11865

OpenRA Project. (n.d.). *About OpenRA*. https://www.openra.net/about/

OpenRA-RL Contributors. (2026). *OpenRA-RL: Command AI to play Red Alert*. https://openra-rl.dev/

Ramos-Silva, J. N., & Burke, P. J. (2026). *A universal large language model: Drone command and control interface*. arXiv:2601.15486. https://arxiv.org/abs/2601.15486

Shao, X., Jiang, W., Zuo, F., & Liu, M. (2024). *SwarmBrain: Embodied agent for real-time strategy game StarCraft II via large language models*. arXiv:2401.17749. https://arxiv.org/abs/2401.17749

Vinyals, O., Babuschkin, I., Czarnecki, W. M., Mathieu, M., Dudzik, A., Chung, J., Choi, D. H., Powell, R., Ewalds, T., Georgiev, P., Oh, J., Horgan, D., Kroiss, M., Danihelka, I., Huang, A., Sifre, L., Cai, T., Agapiou, J. P., Jaderberg, M., & Silver, D. (2019). Grandmaster level in StarCraft II using multi-agent reinforcement learning. *Nature, 575*(7782), 350-354. https://doi.org/10.1038/s41586-019-1724-z

Wang, G., Xie, Y., Jiang, Y., Mandlekar, A., Xiao, C., Zhu, Y., Fan, L., & Anandkumar, A. (2023). *Voyager: An open-ended embodied agent with large language models*. arXiv:2305.16291. https://arxiv.org/abs/2305.16291